

IMPLEMENTATION GUIDE

Lane Detection Prototype

A step-by-step path from an empty folder to a working web demonstration

Research Model, Video Pipeline and Streamlit Website

1. What Will Be Built?

The final prototype is a website that accepts a forward-facing road video and displays detected lane boundaries. The proposed model uses recent frames to make predictions more stable when markings are faded, broken or temporarily hidden. It also reports confidence instead of pretending that every prediction is reliable.

Final user experience

Upload video -> select Baseline or Temporal Model -> process -> view lane overlay and measurements -> download processed video and CSV results.

System boundary

Included	Not included
Camera-video lane perception	A complete self-driving vehicle
Lane mask, curves, centre and confidence	Direct steering or braking of a real car
Baseline versus temporal-model comparison	LLM or RAG without a real need
Research evaluation and a demonstration website	Production safety certification

Core technology

- Python and PyTorch for deep-learning development.
- OpenCV for video reading, frame processing and output-video writing.
- A lightweight lane-segmentation baseline, such as TwinLiteNet.
- ConvGRU for short-term video memory in the proposed model.
- Streamlit for the website interface.
- Pandas and Matplotlib for results, tables and research figures.

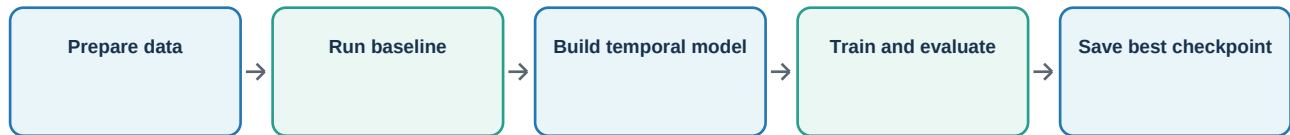
Definition of success

A user can upload a road video, obtain a playable output with left/right lanes, lane centre and confidence, compare the baseline with the temporal model, and download the result. The team can reproduce the reported accuracy, stability and speed measurements on a fixed test set.

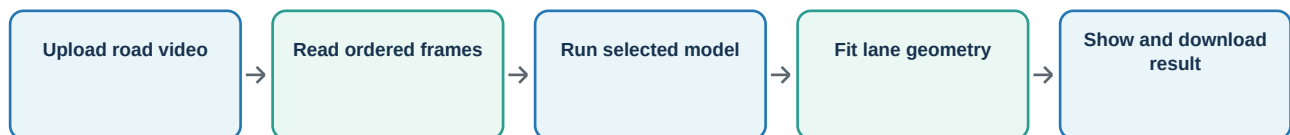
2. Complete Project Flow

The work is divided into model development and website inference. Training happens outside the website. The website only loads a saved checkpoint and processes uploaded videos.

Development flow



Website runtime flow



Why training and the website are separate

Training may take hours and normally needs a GPU. Website inference should start quickly, use fixed weights and give a repeatable result. The website therefore loads `best_model.pth`; it does not retrain the network.

Exact development order

Step	Deliverable
1	Empty Streamlit website with video upload
2	Pretrained baseline working on one image
3	Baseline processing a complete video
4	Baseline metrics and failure examples recorded
5	Sequence dataset loader and ConvGRU model
6	Temporal training, evaluation and ablation
7	Confidence and lane-geometry post-processing
8	Final website, downloadable output and deployment
9	Research tables, figures and paper draft

3. Environment and Folder Setup

Install Python 3.10 or 3.11, Git and Visual Studio Code. Create a separate virtual environment. Install PyTorch using the command generated by the official PyTorch installation selector because CPU and GPU commands differ.

Commands

```
mkdir lane-temporal-project
cd lane-temporal-project
python -m venv venv

# Windows Command Prompt
venv\Scripts\activate

# macOS or Linux
source venv/bin/activate

python -m pip install --upgrade pip
pip install streamlit opencv-python numpy pillow pandas matplotlib
```

Recommended folder structure

```
lane-temporal-project/
|-- app.py
|-- requirements.txt
|-- README.md
|-- configs/          # experiment settings
|-- data/             # loaders and split files
|-- models/           # baseline, encoder, ConvGRU and decoder
|-- training/         # losses and training loop
|-- inference/        # frame preprocessing, video loop and overlays
|-- evaluation/       # metrics, comparisons and plots
|-- weights/best_model.pth
|-- outputs/          # videos, CSV files and figures
```

First test

Create **app.py** with a title and file uploader, then run **streamlit run app.py**. This milestone is complete when the browser opens and an uploaded video plays.

Version-control rule

Commit code, configuration and documentation to Git. Do not commit large datasets, temporary videos, private files or unnecessary training checkpoints.

4. Integrate a Single-Frame Baseline

Begin with a working published baseline before creating the temporal model. TwinLiteNet is a practical choice because it is lightweight and includes lane-segmentation code. It is a reference implementation, not the new research contribution.

Installation

```
git clone https://github.com/chequanghuy/TwinLiteNet.git baseline
cd baseline
pip install -r requirements.txt
```

Baseline checklist

- Download the checkpoint specified by the repository and verify its source.
- Run the original inference example without modifying the architecture.
- Confirm input resolution, BGR-to-RGB conversion, normalization and tensor shape.
- Identify which output channel represents lanes and how the mask is thresholded.
- Save one original image, its probability map, binary mask and overlay.
- Record package versions and the exact commit or release used.

Create a clean wrapper

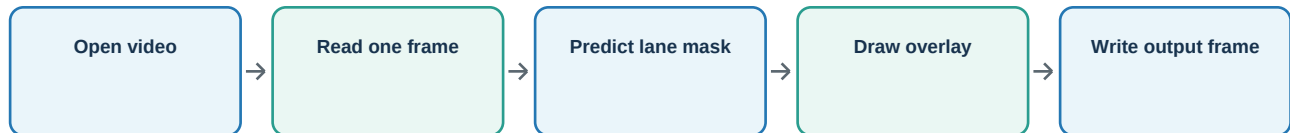
```
def predict_lane(frame_bgr):
    tensor = preprocess(frame_bgr)
    with torch.no_grad():
        output = model(tensor)
        probability = select_lane_probability(output)
        mask = postprocess(probability)
    return probability, mask
```

Exit condition

Do not continue until the baseline correctly processes an unseen road image through one reusable `predict_lane(frame)` function.

5. Convert Image Inference into Video Inference

A video is processed as an ordered sequence of images. OpenCV reads one frame at a time, sends it to the model, draws the result and writes the annotated frame into a new video.



Processing skeleton

```
cap = cv2.VideoCapture(input_path)
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
writer = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc("mp4v"),
    fps, (width, height))

while True:
    ok, frame = cap.read()
    if not ok:
        break
    probability, mask = predict_lane(frame)
    overlay = draw_lane_overlay(frame, mask)
    writer.write(overlay)

cap.release()
writer.release()
```

Required checks

- The output opens in a normal video player and has the expected duration.
- Frames are not stretched, rotated, duplicated or written in the wrong color order.
- FPS and per-frame latency are measured using the same timing method for every model.
- The loop releases VideoCapture and VideoWriter even when processing fails.
- The baseline output is saved before any temporal smoothing is added.

6. Prepare Video Sequences and Labels

The proposed model needs consecutive frames from the same video. Start with a sequence length of three. A sample window contains frames $t-2$, $t-1$ and t , while the target is normally the lane mask for frame t .

Data-loader output

Item	Form	Example
Images	[batch, time, channels, height, width]	[4, 3, 3, 360, 640]
Target	Current-frame binary lane mask	[4, 1, 360, 640]
Metadata	Video ID, frame IDs and condition	video_07, 104-106, shadow

Correct splitting

- Assign complete videos or routes to train, validation or test before creating windows.
- Never randomly distribute adjacent frames across different splits; this causes leakage.
- Keep windows in chronological order and never shuffle frames inside one window.
- Apply the same crop, flip or geometric transform to every frame and mask in a sequence.
- Keep an untouched real test set; synthetic shadows or rain are training support, not proof of real-weather robustness.

Recommended starting datasets

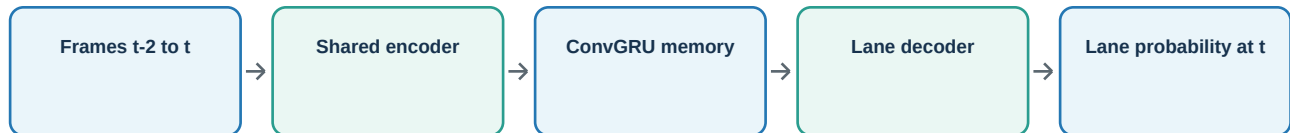
Use VIL-100 when sequence-based lane labels are required. CULane can support difficult lane scenes and baseline comparison. IDD or a carefully annotated small Indian-road video set can be used for domain testing, but only when the lane ground truth is defensible.

Loader exit condition

Print several batches and visually verify that every window is consecutive, belongs to one video and matches the correct target mask.

7. Build and Train the Temporal Model

The encoder extracts visual features from each frame. A ConvGRU combines the current features with short-term memory. The decoder converts the final memory state into the current lane-probability map.



Forward logic

```
hidden = None
for frame in ordered_frames:
    features = encoder(frame)
    hidden = convgru(features, hidden)
    lane_probability = decoder(hidden)
return lane_probability
```

Training strategy

- Initialize the encoder from the working baseline and initially freeze or partly freeze it.
- First overfit a very small sample. If the model cannot learn it, debug the pipeline before full training.
- Use a combination of segmentation loss and carefully designed continuity/temporal losses.
- Reset hidden memory when a new video begins. Do not carry memory between unrelated scenes.
- Save the best validation checkpoint, configuration, seed, epoch logs and environment details.
- Compare sequence lengths 1, 3 and 5 and report both accuracy and latency.

Conceptual objective

Total loss

$L_{total} = L_{segmentation} + \lambda_c \times L_{continuity} + \lambda_t \times L_{temporal}$. Select lambda values on validation data and show an ablation for each added component.

Important caution

A moving camera changes the pixel position of a lane between frames. Temporal consistency should therefore use aligned features/masks where possible and must not force neighbouring predictions to be identical.

8. Add Lane Geometry and Confidence

The neural network produces lane probabilities. Post-processing converts them into compact lane information that can be shown on the website or passed to a future simulator.

Geometry flow



- Restrict analysis to a useful road region and remove tiny disconnected components.
- Separate plausible left and right points by position and continuity.
- Fit a polynomial or spline with outlier rejection; reject crossing lanes and impossible width changes.
- Calculate the lane centre from both valid boundaries. Mark one-sided estimates as partial.
- Treat pixel-space offset and curvature as demonstration values. Metres require camera calibration and perspective mapping.

Confidence signals

Signal	Purpose
Mean lane probability	Measures visual evidence from the network
Curve continuity	Penalizes fragmented or very short detections
Temporal agreement	Checks whether movement from recent frames is plausible
Geometric validity	Checks ordering, width and curvature constraints

Convert the combined score into High, Medium and Low only after selecting thresholds on validation data. Use Expected Calibration Error or a reliability diagram to test whether stated confidence matches actual correctness.

Safe failure rule

If lane evidence is too weak or memory is too old, return 'no reliable lane'. The model may bridge a short occlusion, but it must not invent a lane indefinitely.

9. Build the Streamlit Website

The website is a demonstration layer. It loads a saved model once, accepts a video, runs the same tested inference function and displays the output. Keep command-line inference available so scientific experiments do not depend on the interface.

Basic application

```
import streamlit as st

st.set_page_config(page_title="Lane Detection Demo", layout="wide")
st.title("Temporal Lane Detection Prototype")
uploaded = st.file_uploader("Upload a road video",
                             type=["mp4", "avi", "mov"])
if uploaded is not None:
    st.video(uploaded)
```

Inference integration

```
@st.cache_resource
def load_model():
    return LaneModel(weights="weights/best_model.pth")

if uploaded and st.button("Process video"):
    input_path = save_temporary_upload(uploaded)
    output_path, metrics = process_video(input_path, load_model())
    st.video(output_path)
    st.metric("Average confidence", metrics["confidence"])
    st.metric("Average FPS", metrics["fps"])
```

Interface contents

- File uploader with a clear size and duration limit.
- Baseline/Temporal Model selector and an explicit Process button.
- Progress bar, processing time and useful error messages.
- Original and processed video, confidence, FPS and condition notes.
- Download buttons for the processed MP4 and optional per-frame CSV.
- A visible disclaimer that this is a research prototype, not a safety-certified control system.

10. Evaluation Before Deployment

An attractive video is not enough for a research project. Evaluate every method on the same test videos and report whether temporal memory improves accuracy and stability without unacceptable computational cost.

Required measurements

Metric	What it demonstrates
Precision, Recall and F1	Correctness and recovery of lane pixels or lane instances
Lane IoU	Overlap between prediction and ground truth
Continuity or gap score	Degree of fragmentation in detected lanes
Temporal stability	Amount of aligned flicker between consecutive frames
ECE/reliability diagram	Whether confidence matches observed correctness
FPS and latency	Processing speed on declared hardware
Parameters/FLOPs	Additional computational cost of the proposed method

Minimum comparison table

- Single-frame baseline.
- Baseline plus ConvGRU.
- Baseline plus continuity/temporal losses.
- Complete model with ConvGRU, losses and confidence.
- Sequence lengths 1, 3 and 5.
- Condition-wise results: normal, curve, shadow, night and occlusion when labelled data is available.

Research decision

Keep the proposed method only if it shows a repeatable benefit over the baseline. If it improves continuity but reduces accuracy or speed, report that trade-off honestly and revise the design.

11. Deployment and Completion Checklist

Test the website locally first. Then push the verified source and requirements file to GitHub. Streamlit Community Cloud, Hugging Face Spaces or Render may be used for demonstration, subject to their current resource and upload limits.

Deployment preparation

- Generate requirements.txt from a clean working environment and test installation again.
- Use `model.eval()` and `torch.no_grad()` during inference.
- Cache model loading with `st.cache_resource`, not video-specific output.
- Limit video duration/resolution so CPU-only hosting remains usable.
- Handle invalid formats, missing codecs, processing exceptions and temporary files.
- Test the deployed link from another device and browser.

Final milestone checklist

Status	Milestone	Required evidence
☐	Website shell	Upload and playback work
☐	Baseline image	Mask and overlay saved correctly
☐	Baseline video	Complete playable output and timing log
☐	Sequence pipeline	Correct ordered batches and video boundaries
☐	Temporal checkpoint	Training logs and best validation model
☐	Fair evaluation	Metrics, ablations and failure examples
☐	Final website	Model selection, output video, metrics and downloads
☐	Deployment	Working link tested outside developer machine
☐	Research package	Code version, tables, figures and limitations frozen

Project completion

The prototype is complete when a user can upload a road video, select the baseline or temporal model, receive and download a processed video with lane geometry and calibrated confidence, and trace the displayed results to reproducible experiments.

12. Starting Resources

Resource	Purpose	Link
PyTorch installation	Select the correct CPU/GPU command	https://pytorch.org/get-started/locally/
OpenCV video tutorial	Read and write video frames	https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
Streamlit documentation	Build and deploy the website	https://docs.streamlit.io/
TwinLiteNet	Lightweight single-frame baseline	https://github.com/chequanghuy/TwinLiteNet
VIL-100 / MMA-Net	Video lane dataset and baseline	https://github.com/yujun0-0/MMA-Net
CULane	Challenging lane-detection dataset	https://xingangpan.github.io/projects/CULane.html
IDD	Unstructured Indian driving scenes	https://idd.insaan.iiit.ac.in/

Common mistakes to avoid

- Starting with temporal training before baseline inference and metrics work.
- Training inside the Streamlit website.
- Mixing adjacent frames across train and test splits.
- Adding LLM or RAG simply to make the project sound complex.
- Claiming real-time speed without naming the hardware, resolution and timing method.
- Using only attractive output videos as research evidence.
- Calling pixel measurements metres without camera calibration.
- Claiming novelty before an updated literature and patent search.

Recommended immediate action

Begin here

Create the repository and virtual environment, open a basic Streamlit uploader, and run the unmodified baseline on one image. This produces the fastest reliable foundation for every later step.